

Load Balancer From Scratch

Advanced Cloud Computing — Lab Assignment 1

Fall 2026

Abstract

This assignment introduces you to a major cloud computing provider's services and to the deployment of your own services on top of them. You will practice Infrastructure as Code (IaC) and use AWS APIs and SDKs to create and deploy a cluster of virtual machines. You will then design, implement, and deploy your own application load balancer, and use it to manage traffic across your cluster. Finally, you will use the metrics provided by the cloud provider to analyze and compare the performance of your cluster configurations.

1 Introduction

Amazon Web Services (AWS) remains the leading infrastructure-as-a-service cloud provider. Amazon Elastic Compute Cloud (Amazon EC2) is a web-based service that lets you run application programs as virtual machines (VMs) inside the AWS public cloud. Amazon EC2 lets a developer provision compute capacity for a wide range of workloads, and scale that capacity up or down within minutes, either through the AWS Management Console, the AWS CLI, or an SDK. A developer can also define auto-scaling policies and groups to manage a fleet of instances automatically.

During the first laboratory session, the instructor will present an introduction to cloud computing and to AWS EC2, along with the guidelines needed to create instances and connect to them over SSH. Once you are comfortable with the provider, you will build a small cluster of instances and begin benchmarking it under different configurations.

Note: Stop or terminate your EC2 instances whenever you are not actively using them. Idle running instances are the most common source of wasted cloud credits.

Your team will be provisioned with a cloud budget for this and the following assignments in the course, most commonly through an AWS Academy Learner Lab or an equivalent classroom sandbox provided by your institution (AWS no longer issues credit grants directly through the legacy AWS Educate program). Manage your budget carefully — Learner Lab sessions also have a maximum session length and a hard budget cap, after which your lab is locked for the remainder of the term, so plan your working sessions accordingly.

The overall goals of this lab assignment are:

- Create clusters using EC2 and an Elastic Load Balancer (ELB).
- Benchmark clusters to compare their performance across instance families and generations.
- Develop your own application load balancer.
- Automate your entire solution using Infrastructure as Code.

- Report your findings in a handover-documentation format.
- Demonstrate your work in a short live demo.

2 What Is an EC2 Instance?

An Amazon EC2 instance is a virtual server running in Amazon’s cloud. It provides scalable compute capacity so you can run applications and services without owning physical hardware. Instances are grouped into families that are optimized for different workloads: general purpose, compute-optimized, memory-optimized, storage-optimized, and accelerated-computing (GPU) instances. You can launch, configure, resize, and terminate instances on demand and pay only for what you use.

2.1 Instance Generations and the Graviton Option

AWS instance families are versioned (t2, t3, t3a, t4g, m6i, m7g, c7g, and so on). Newer generations run on the Nitro hypervisor and generally deliver better price-performance than their predecessors, so unless you have a specific reason to use an older generation, you should default to the current generation available in your region. AWS also offers Graviton (Arm-based, suffix “g”) instances such as t4g or m7g, which typically offer noticeably better price-performance than their x86 equivalents for workloads — like a small Python/FastAPI service — that don’t depend on x86-only binaries.

For this assignment, you are free to benchmark both an x86 family (e.g., t3) and a Graviton family (e.g., t4g) if you want an extra point of comparison for your report, but it is not required — the mandatory comparison is between a burstable, low-capacity family and a larger, sustained-performance family (see Section 6).

3 What Is a Load Balancer?

Elastic Load Balancing (ELB) is AWS’s managed load-balancing service. It automatically distributes incoming application traffic across multiple targets and scales with traffic demand, helping your team adjust capacity to match incoming load. AWS currently offers four ELB types:

- **Application Load Balancer (ALB)** — operates at Layer 7 (HTTP/HTTPS). Supports path- and host-based routing, and can route requests to one or more ports per target, including dynamic port mapping for containerized targets.
- **Network Load Balancer (NLB)** — operates at Layer 4 (TCP/UDP/TLS) and can handle very high, volatile request rates with ultra-low latency, using a flow-hash algorithm to pick a target and forwarding the connection without modifying its headers.
- **Gateway Load Balancer (GWLB)** — routes and inspects traffic transparently at Layer 3/4, typically used in front of third-party virtual appliances (firewalls, intrusion-detection systems); included here for completeness, but out of scope for this assignment.
- **Classic Load Balancer (CLB)** — the legacy Layer 4/Layer 7 balancer. AWS treats CLB as a legacy offering: it requires a fixed mapping between the load-balancer port and the target’s instance port, does not support dynamic port mapping, and is not recommended for new workloads. Use it only if your assignment explicitly asks you to compare it against an ALB.

For this assignment, you will build a simple application load balancer of your own, in addition to using AWS’s managed load balancer for comparison. Given a target group, your custom load balancer will periodically probe each instance (by sending requests and measuring response time, or by reading a CloudWatch metric), keep track of which instance is currently the most responsive, and route new requests to that instance.

4 AWS CloudWatch

Elastic Load Balancing publishes metrics to Amazon CloudWatch for your load balancers and their targets. CloudWatch lets you retrieve these metrics as time-series data: each data point has a timestamp and, optionally, a unit. You can use metrics to confirm your system is behaving as expected — for example, tracking the number of healthy targets behind a load balancer over time — and you can attach a CloudWatch alarm to a metric to trigger an action (such as a notification) whenever the metric moves outside an acceptable range.

Browse the available metrics here: <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-cloudwatch-metrics.html>

5 Benchmarking

5.1 FastAPI App

You will deploy a small FastAPI application on every EC2 instance. Each instance must be identifiable by a unique ID: for example, instance 1 should respond with a message such as *Instance number 1 is responding now!*. Keep the web service implementation as simple as possible — the goal of this assignment is the infrastructure and the load-balancing logic, not the application itself.

FastAPI documentation: <https://fastapi.tiangolo.com/>

5.2 Load Balancer Components

There are two acceptable ways to implement your application load balancer:

- **Active probing:** your load balancer sends requests to each EC2 instance in its target group, measures response time, and keeps track of which instance is currently more responsive. It routes new requests to that instance until another instance becomes more responsive.
- **CloudWatch-driven:** your load balancer polls a CloudWatch metric for each instance (e.g., `CPUUtilization`, or a custom metric you publish yourself). Once an instance’s metric crosses a threshold you define, new requests are routed to another instance in the target group.

6 Setup

6.1 Running Your EC2 Instances via the AWS Academy Learner Lab

Before you can provision or connect to any EC2 instance, you must launch and start your team’s AWS Academy Learner Lab session from the course platform. Figures 1 and 2 show the two steps below.

1. **Open the lab launcher.** In the course platform, click **Courses** in the left sidebar, then **Modules**. Scroll down to the **AWS Academy Learner Lab** module and click **Launch AWS Academy Learner Lab** (Figure 1).

2. **Start the lab session.** On the page that opens (Figure 2), click **Start Lab**. Wait for the status dot next to the **AWS** button to turn green — this means your temporary sandbox account is ready and billing against your \$50 budget has begun.
3. **Open the console or use the built-in terminal.** Once the indicator is green, click the **AWS** button to open the Management Console, already signed in, in a new tab — this is where you can launch EC2 instances by hand to explore the console. For the automated workflow this assignment requires, use the terminal embedded directly in the page instead: it ships with the AWS CLI already configured with your temporary credentials, so commands such as `aws ec2 run-instances ...` work immediately, with no `aws configure` step.
4. **Monitor your budget and time.** The **Used \$X of \$50** counter and the session timer at the top of the page (Figure 2) track your spend and elapsed session time in real time — check them before kicking off a benchmarking run so you don't get cut off mid-experiment.
5. **End the session when you're done.** Click **End Lab** to stop billing. Check the **Readme** panel on the right of the page for whether your EC2 instances are stopped or terminated between sessions before you end one you plan to resume later.

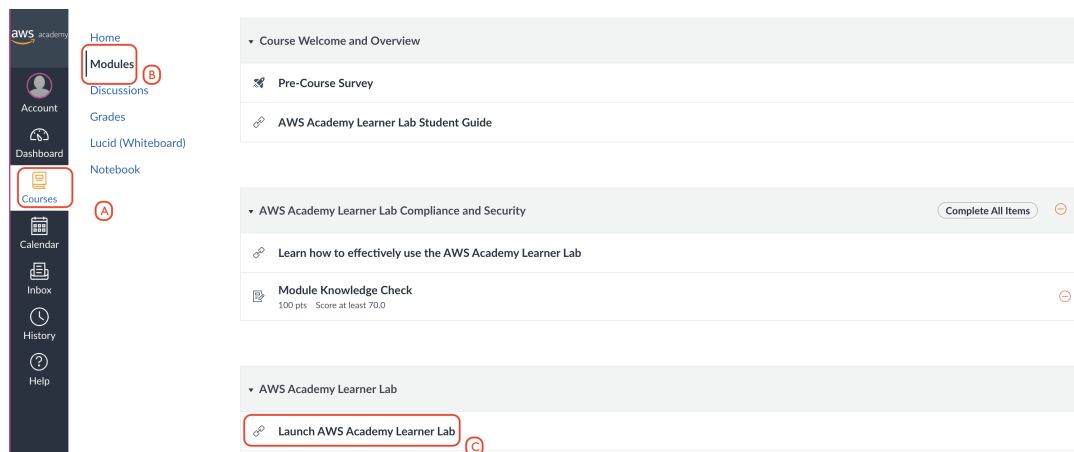


Figure 1: Locating the lab launcher: **Courses** → **Modules** → the **AWS Academy Learner Lab** module → **Launch AWS Academy Learner Lab**.

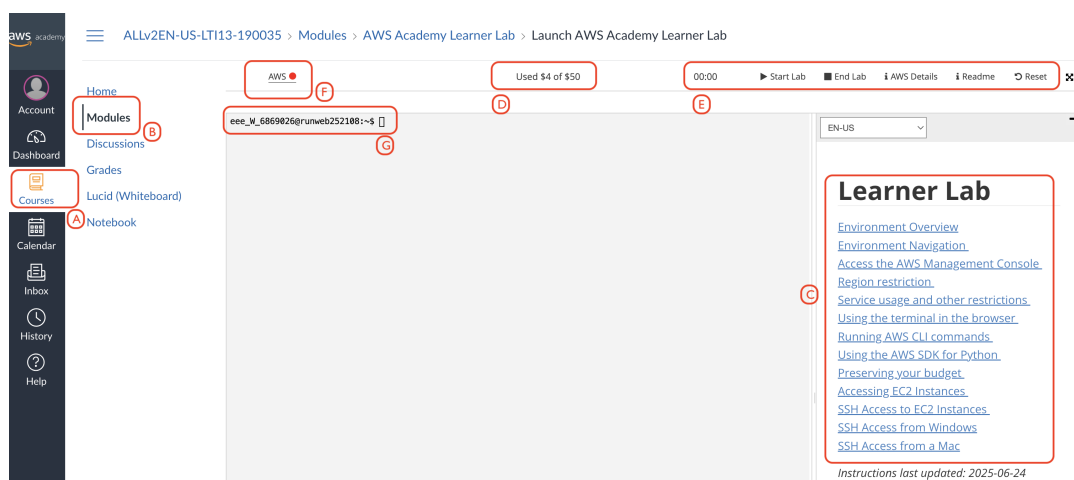


Figure 2: The Learner Lab environment after launch: budget used (D), session timer and Start/End Lab controls (E), the AWS console button (F), and the embedded, pre-authenticated terminal (G) for running AWS CLI commands directly.

Note: Keep an eye on the **Used \$X of \$50** indicator near the top of the page throughout the term — this is the same budget referenced in Section 1, and it does not reset between sessions. Also check the **Region restriction** panel on the right-hand side of the lab page: AWS Academy Learner Labs are normally locked to a single AWS region, which is why the note below lists fallback instance families in case your assigned family isn’t offered there.

When you are done for the session, click **End Lab** to stop billing. Read the **Readme** panel on your own lab page for the exact behavior of your account between sessions (for example, whether running instances are stopped or terminated) before you end a session you plan to resume later — this varies and directly affects whether your cluster will still be there the next time you start the lab.

Before you begin the rest of the setup below:

- Provision 9 EC2 instances: 5 burstable, low-capacity instances (`t3.micro`) and 4 larger, sustained-performance instances (`m7g.large` or `c7g.large`, depending on availability in your region).
- Deploy the FastAPI app from Section 5.1 on each instance.

Note: `t3.micro/t3.large` replace the older `t2` family referenced in previous editions of this lab — `t3` is current-generation, Nitro-based, and typically ~10% cheaper at equivalent size. If `t3` or the Graviton `m7g/c7g` families are unavailable in your assigned region, `m6i.large` or `c6i.large` are acceptable x86 substitutes for the “large” cluster; note the substitution and the reason for it in your report.

6.2 Clustering and Load Balancer Setup

Create two separate clusters: one built from your five `t3.micro` instances, and one built from your four larger instances. You will send requests to each cluster independently and measure the responses using CloudWatch metrics.

Configure your instances so that the small-instance cluster listens on the `/cluster1` route and the large-instance cluster listens on the `/cluster2` route; each route should return that instance’s unique ID. Sending traffic to `/cluster1` should only ever produce responses from the small-instance cluster, and traffic to `/cluster2` should only ever produce responses from the large-instance cluster.

Create two target groups, one per cluster, and attach both to a single Application Load Balancer with two listener rules — one per target group — so that path-based routing sends `/cluster1` traffic to the small-instance target group and `/cluster2` traffic to the large-instance target group.

Follow AWS’s current guide to create an Application Load Balancer: <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/create-application-load-balancer.html>

6.3 Deploying Your FastAPI App on Each Instance

Update the OS packages on each instance:

```
# Amazon Linux 2023
sudo dnf update -y

# Ubuntu 24.04 LTS
```

```
sudo apt-get update && sudo apt-get upgrade -y
```

Install Python 3 and pip:

```
# Amazon Linux 2023
sudo dnf install python3 python3-pip -y

# Ubuntu 24.04 LTS
sudo apt-get install python3 python3-pip python3-venv -y
```

Note: Both Amazon Linux 2023 and Ubuntu 24.04 ship a system Python that is “externally managed” and will refuse a bare `pip install`. Create a virtual environment first (`python3 -m venv .venv && source .venv/bin/activate`) rather than passing `--break-system-packages` to pip, so your dependencies stay isolated from the OS packages.

Transfer your Python app to the instance with `scp`:

```
scp -i /path/to/private_key.pem /path/to/local/main.py \
ec2-user@<instance_public_ip>:/path/to/destination
```

Replace `ec2-user` with `ubuntu` if you are running an Ubuntu AMI. The private key is the local path to your key pair, and `instance_public_ip` is the instance’s public IPv4 address (or DNS name).

A FastAPI starter template will be shared on the course platform. Install the dependencies inside your virtual environment and run the app:

```
source .venv/bin/activate
pip install fastapi "uvicorn[standard]"
uvicorn main:app --host 0.0.0.0 --port 8000
```

Note: For a demo-grade deployment, run `uvicorn` under a systemd unit (or `nohup ... &`) so the process survives your SSH session ending, and remember to open port 8000 (or whichever port you choose) in the instance’s security group.

6.4 Benchmarking

Send one thousand simultaneous requests to each target group and measure their response times. You may use a hand-rolled `asyncio/aiohttp` script such as the one below, or an established load-testing tool (Locust, k6, or hey) if your team prefers — note your choice and justify it briefly in your report.

```
import asyncio
import aiohttp
import time

async def call_endpoint_http(session, request_num):
    url = "http://<your-load-balancer-dns>/cluster1"
    headers = {"content-type": "application/json"}
    try:
        async with session.get(url, headers=headers) as response:
            status_code = response.status
            response_json = await response.json()
```

```

        print(f"Request {request_num}: Status Code: {status_code}")
        return status_code, response_json
    except Exception as e:
        print(f"Request {request_num}: Failed - {str(e)}")
        return None, str(e)

async def main():
    num_requests = 1000
    start_time = time.time()

    async with aiohttp.ClientSession() as session:
        tasks = [call_endpoint_http(session, i) for i in range(num_requests)]
        await asyncio.gather(*tasks)

    end_time = time.time()
    print(f"\nTotal time taken: {end_time - start_time:.2f} seconds")
    print(f"Average time per request: {(end_time - start_time) / num_requests:.4f}
↪ seconds")

if __name__ == "__main__":
    asyncio.run(main())

```

6.5 Team Seed (Personalization)

This section exists to make sure the work you submit is actually this term's work, done by your team. You will derive a small numeric *team seed* from your student IDs and thread it through several concrete, checkable values in your deployment. Reusing a previous year's solution, or another team's, without recomputing these values will be immediately visible in your live responses, your logs, and your report.

Compute your seed as follows: sort your team's student IDs in ascending order, join them with hyphens into a single string, and hash it:

```

import hashlib

student_ids = sorted(["idA", "idB", "idC"]) # your team's IDs, as strings
joined = "-".join(student_ids)
seed = int(hashlib.sha256(joined.encode()).hexdigest(), 16) % 10000

```

Using this seed, your deployment must:

- Have every instance's FastAPI response include the seed, e.g. as an extra `team_seed` field in the JSON body *and* as a custom `X-Team-Seed` response header.
- Set your custom load balancer's failover threshold to $50 + (\text{seed} \bmod 200)$ milliseconds, instead of a value you pick arbitrarily. State this computed threshold explicitly in your report, alongside the seed itself.
- Name your top-level automation script or IaC stack `assignment1-team-<seed>` so your infrastructure is traceable to your team seed on sight.

Note: We will recompute a sample of teams' seeds from the official course roster and compare them against what appears in your code, your live responses, and your report. A missing, hardcoded, or mismatched seed — or a seed that matches a previously submitted or publicly available solution — will be treated as an academic integrity concern, not a deduction.

7 Automation and Infrastructure as Code

Your solution must be end-to-end automated: creating the instances, the load balancer, and the target groups; transferring and starting the FastAPI app; and running the benchmark script should all be triggerable from a single command during your demo.

You may use any of the following approaches, depending on your team's preference and experience:

- **AWS CLI + shell script** — the most direct path; wrap the `aws ec2`, `aws elbv2`, and `aws cloudwatch` CLI commands in a single bash script.
- **An AWS SDK** (boto3 for Python, the JavaScript/TypeScript SDK v3, etc.) driven from a small provisioning script.
- **A declarative IaC tool** such as Terraform, AWS CloudFormation, or the AWS Cloud Development Kit (CDK) — recommended if your team wants practice with tools commonly used in industry.

AWS SDKs reference: <https://aws.amazon.com/developer/tools/>

8 Live Verification: The Chaos Check

Once your automated deployment finishes on demo day, the instructor or a TA will, without advance notice, stop or terminate one running instance in *each* of your two clusters. Live, in front of the instructor, your team must show:

1. Which instance was affected, identified by its unique ID and team seed.
2. That your custom load balancer detects the failure and stops routing to that instance within a reasonable window (roughly 30 seconds), demonstrated with a continuously running `curl` loop or your own dashboard.
3. That the affected cluster keeps serving correct responses from its remaining instances throughout the disruption.
4. The relevant CloudWatch graphs (target health, requests per target) spanning the moment of failure, timestamped during your actual demo slot — not a screenshot from an earlier test run.

Note: This check happens only at demo time and cannot be prepared in advance from a repository alone — passing it requires your infrastructure and your load balancer to be live, correctly wired, and actually able to react, at the exact moment of the test. A submission whose infrastructure is not live and independently operable during its scheduled demo slot cannot receive credit for this section, regardless of what the submitted ZIP file contains.

9 Working in Groups

You should work in groups of two, three, or four students.

10 Report and Submission

One submission per group is required. Write your lab report using the same LaTeX (or equivalent) conventions illustrated by this document, and answer the following in your report:

- Your FastAPI deployment procedure.
- Your cluster setup using the Application Load Balancer, including your two target groups and routing rules.
- The results of your benchmark, including a comparison between the small-instance and large-instance clusters, and between the AWS-managed ALB and your own custom load balancer.
- Your team seed, how it was computed, and the resulting failover threshold (Section 6.5).
- What happened during your live chaos check (Section 8): detection time, behavior of the remaining instances, and timestamped CloudWatch graphs from that demo slot.
- Instructions to run your code end-to-end.

Submit a single ZIP file named `assignment1.zip` containing:

- `report.pdf` — the names of all team members and the full report; you may design the layout as you see fit.
- **Your code** — everything needed to execute and reproduce your benchmark results, with enough comments and documentation for a reader unfamiliar with your setup to follow along.
- `contribution_log.xlsx` — your completed Team Contribution Tracker (see Section 11), exported from the companion spreadsheet distributed alongside this assignment.

11 Marking Rubric

This lab is worth 100 points, split across the criteria below. Partial credit is prorated within each row based on how many of that section’s required elements (as described earlier in this document) are met.

11.1 Individual Contribution Adjustment

One grade is computed per team using the rubric above. Each team member’s *individual* grade is then scaled by a contribution factor derived from that team’s completed Team Contribution Tracker (the companion spreadsheet distributed with this assignment):

- Every logged hour must be attributed to a team member and a rubric criterion above; the tracker totals each member’s logged hours automatically.
- Each member’s *equal share* is $100\%/n$ for a team of n members. Their *contribution factor* is their share of logged hours divided by the equal share, clamped to the range $[0.85, 1.15]$ so that no single log entry can swing a grade drastically.
- Your individual grade is team grade $\times$ contribution factor, capped at 100 points.

Criterion	Weight	What full credit looks like
Cluster setup & FastAPI deployment (Sec. 6)	5%	All 9 instances provisioned with the correct families, FastAPI running and reachable on each, unique instance IDs returned correctly.
AWS ALB configuration (Sec. 6.1)	10%	Two target groups, correct listener rules, <code>/cluster1</code> and <code>/cluster2</code> each route only to their intended cluster.
Custom load balancer (Sec. 3, 5.2)	20%	Correctly implements active probing or CloudWatch-driven routing, measurably redirects traffic toward the more responsive instance; code is understandable and matches what the report describes.
Team seed compliance (Sec. 6.5)	5%	Seed correctly computed and reproducible from the roster; header, failover threshold, and stack name all consistently derived from it.
Benchmarking methodology & results (Sec. 5, 6.3)	15%	1,000-request benchmark correctly run against both clusters and both load balancers, results are sound, and the report draws a clear, correct comparison.
Automation / Infrastructure as Code (Sec. 7)	15%	A single command reliably provisions instances, load balancer, target groups, deploys the app, and runs the benchmark end-to-end with no manual steps.
Live chaos check (Sec. 8)	10%	Failure is correctly detected and routed around within the expected window; remaining instances keep serving correctly; live evidence (not pre-recorded) matches the report.
Report quality & completeness (Sec. 10)	15%	All required questions answered clearly, findings supported by the actual data collected, well organized and easy to follow.
Code quality & documentation	5%	Code is readable, reasonably organized, and commented well enough that a TA unfamiliar with the project can follow and rerun it.

Note: Keep the tracker updated *as you go* rather than reconstructing it the night before submission — timestamps that are all identical, or that cluster suspiciously around the deadline, will prompt us to ask your team to explain your logged effort in person.